

# Concurrent Programming

元培数科 王恩博

Let's get started →

2025/12/17



# 什么是并发编程

## Concurrent Programming

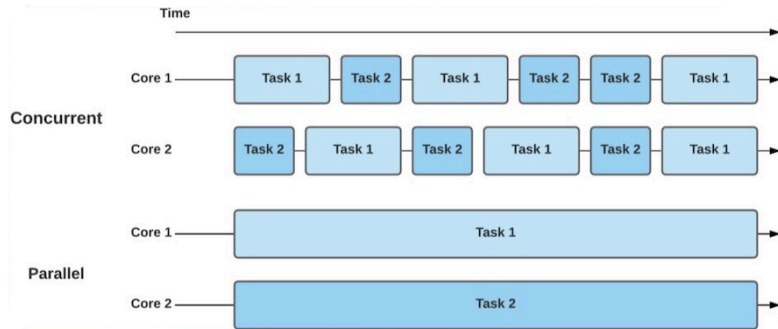
逻辑控制流在时间上重叠，就是 **并发**。

- 内核级并发：进程切换
- 应用级并发：线程切换

本质：**时间片轮转**，“不要让核闲下来。”

并发程序：使用应用级并发的程序。

**并行**：物理上在同一时刻执行多个并发任务，依赖多核处理器等物理设备。



# 顺序 vs 并发

Sequential vs Concurrent

## 顺序

- 多个操作“依次处理”
- 一个操作“处理完毕”后，才能处理下一个操作
- 会导致 **阻塞**

## 并发

- 将一个操作分割成多个部分并允许无序处理
- **并行**：多个操作“同时”处理，要求多核处理器等物理设备
- 单核情况下，并发的效率提升主要来源于减少阻塞，让核的利用率更高，类似于之前 Arch lab 中的“戳气泡”

# 并发编程的实现方法

## Implementation methods

### 基于进程

- 多进程
- 多个进程通过 **内核调度** 实现并发
- 每个进程都有自己的私有地址空间
- **进程间通信** (IPC) 机制：  
管道、信号、共享内存、**套接字**、**信号量**

### 基于 I/O 多路复用

- 单进程
- 状态机
- 使用 `select` **系统调用** 监控多个文件描述符实现并发
- **存在处理优先级**

### 基于线程

- 单进程
- 多线程
- 多个线程通过 **内核调度** 实现并发，但线程切换比进程切换更轻量
- 线程共享进程的地址空间

**进程是资源分配的基本单位，线程是调度的基本单位**



# 基于进程的并发编程

Concurrent based on processes

- 使用 `fork` `exec` `waitpid` 等函数，实现简单
- **并发的本质**：内核（Kernal）自动交错运行多个逻辑流，不需要我们干预，但是进程上下文切换 **非常耗费资源**
- 每个逻辑流都有自己的私有地址空间：共享信息困难
- 逻辑流之间的通信需要使用 **进程间通信**（IPC）机制：管道、信号、共享内存、**套接字、信号量**

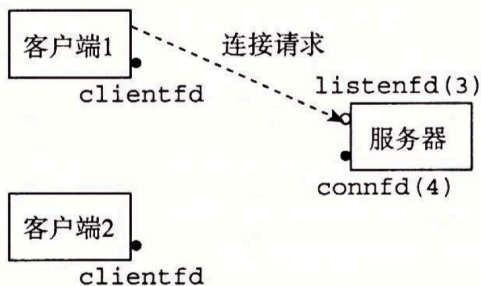


图 12-1 第一步：服务器接受客户端的连接请求

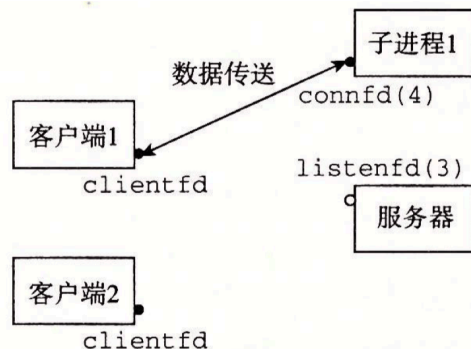


图 12-2 第二步：服务器派生一个子进程为这个客户端服务

# 基于进程的并发编程 - 要点

Concurrent based on processes - Key points

- 必须在父子进程中都适当关闭套接字/描述符（减少引用计数），否则会导致文件描述符泄露
  - 父进程中关闭 `connfd`：不再需要同客户端通信，那是子进程的事情
  - 子进程中关闭 `listenfd`：不再需要监听新的连接，那是父进程的事情
  - 复习：`fork` 之后，父子进程的文件描述符是共享的
- 父进程必须适当使用 `waitpid` 回收子进程，否则会导致僵尸进程
  - 修改 `sigchld_handler`，在其中调用 `waitpid` 回收子进程，每次尽可能回收多个子进程

```
void sigchld_handler(int sig)
{
    while (waitpid(-1, 0, WNOHANG) > 0);
    return;
}

int main() {
    Signal(SIGCHLD, sigchld_handler); // 绑定信号处理函数
    // ...
}
```

回忆：`NOHANG` 表示不阻塞，立即返回，`while` 保证在一次信号处理函数中尽可能回收所有已经退出的子进程

# 基于进程的并发编程 - 要点

Concurrent based on processes - Key points

## 回收资源

看上去好像是一个 `waitpid` 就能“回收资源”，实际上回收这一动作是发生在 `waitpid` 返回到调用它的进程的用户态之前的内核态。

子进程终止时：

1. 子进程终止
2. 内核自动回收大部分资源
3. 保留一些退出信息
4. 进入僵尸状态

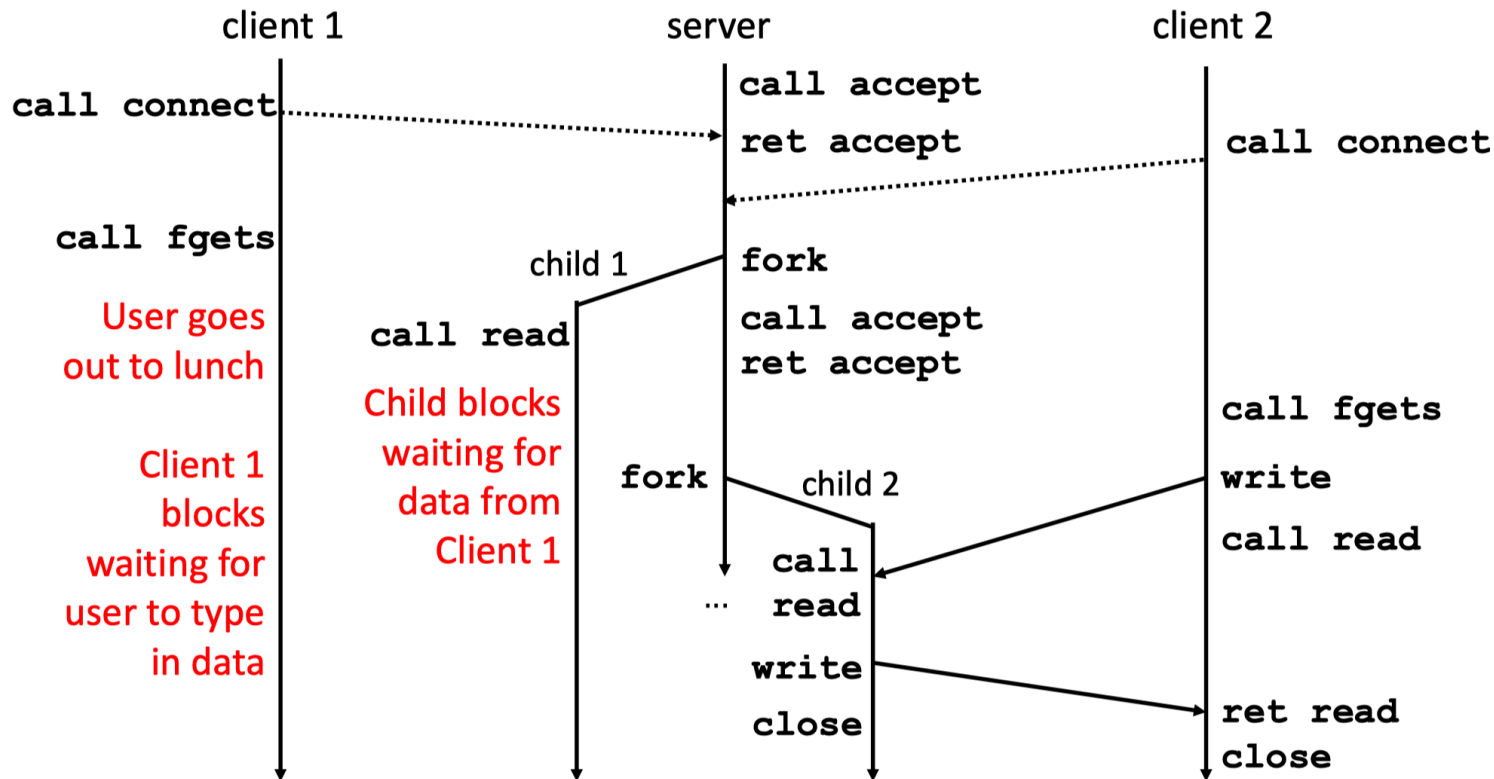
父进程调用 `waitpid` / `wait` 时：

1. (用户态) 父进程调用 `waitpid` / `wait`
2. (内核态) 内核检查子进程状态
3. (内核态) 获取子进程退出信息
4. (内核态) 释放子进程 PCB (Process Control Block, 进程控制块, 内核数据结构, 包含进程的各种信息)
5. (用户态) `waitpid` / `wait` 返回给父进程

\* 对于线程，有类似的机制。

# 基于进程的并发编程

Concurrent based on processes



# 基于进程的并发编程 - 编程实现

```

if (argc != 2) {
    fprintf(stderr, "usage: %s <port>\n", argv[0]);
    exit(0);
}

Signal(SIGCHLD, sigchld_handler);
listenfd = Open_listenfd(argv[1]);
while (1) {
    clientlen = sizeof(struct sockaddr_storage);
    connfd = Accept(listenfd, (SA *)&clientaddr,
    if (Fork() == 0) {
        Close(listenfd); /* 子进程关闭其监听套接字 */
        echo(connfd); /* 子进程为客户端提供服务 */
        Close(connfd); /* 子进程关闭与客户端的连接 */
        exit(0); /* 子进程退出 */
    }
    Close(connfd); /* 父进程关闭已连接的套接字 */
}
}

```

## 注意事项

1. 分叉后，父进程关闭 `connfd`，子进程关闭 `listenfd`
2. 父子进程间不存在对于 `connfd` 的竞争：这个变量是写时复制（COW）的。
3. 大写 `Accept` 函数封装了报错信息，但此时不能封装 `exit` 终止进程。
4. `echo` 函数要处理 `connfd = -1` 的情况。

# 基于进程的并发编程 - 优劣分析

Concurrent based on processes - Advantages and disadvantages

## 1. 进程共享文件表但是不共享用户地址空间，独立的地址空间

- 优点：进程不会覆盖另一个进程的虚拟内存
- 缺点：进程间共享状态信息更加困难（需要使用显式的 IPC 机制）

Linux IPC 机制：管道，套接字，信号量，消息队列，共享内存，屏障，完成变量等

## 2. 基于进程的并发编程 IPC 和进程控制（进程上下文切换）开销较高，比较慢

# 基于 I/O 多路复用的并发编程

Concurrent based on I/O multiplexing

I/O 多路复用是一种同步 I/O 模型，实现同时监视多个文件句柄（套接字/文件描述符）的状态

- 多路：指的是多个网络连接或文件描述符
- 复用：指的是通过一个单一的阻塞对象（如 `select` 系统调用）来监视多个文件描述符的状态变化
  - 没有文件句柄就绪就会阻塞应用程序，交出 CPU，直到有文件句柄就绪
  - **一旦某个文件句柄就绪，就能够通知应用程序进行相应的读写操作**

并发原理：“事件驱动”

1. 原有状况（顺序）：食堂排队排到你，你却迟迟不点餐（充饭卡，选择困难症...），让后面人一直等
2. 多路复用后：类似小程序点餐，谁先准备好点餐就先处理

# 基于 I/O 多路复用的并发编程 - 编程实现

Concurrent based on I/O multiplexing - Programming implementation

```
#include <sys/select.h>
/* fdset 核心: 位向量 */
int select(int n, fd_set *fdset, NULL, NULL, NULL);
/* 返回: 如果有准备好的描述符, 则返回非 0 值标记准备好的描述符个数; 返回 -1 如果出现错误 */
FD_ZERO(fd_set *fdset); /* 全部置零 (清除) */
FD_CLR(int fd, fd_set *fdset); /* 单个置零 (移除对一个文件描述符的监听) */
FD_SET(int fd, fd_set *fdset); /* 设置对于一个文件描述符的监听 */
FD_ISSET(int fd, fd_set *fdset); /* 检查是否在文件描述符内 */
```

- `select` 会 **阻塞**, 直到有文件描述符就绪, 返回值为准备好的文件描述符数量, 即准备好集合的基数
- `select` **返回时会修改** `fdset` (**以供后续调用** `FD_ISSET` **宏**), **因此每次调用** `select` **之前都需要重新设置** `fdset`
- 返回后, 根据代码判断顺序, 存在处理优先级

`select` 函数的参数如下 (后三个参数常设为 `NULL`) :

1. `int nfds` : 指定被监听的文件描述符的范围, 它的值是要监控的所有文件描述符中 **最大值加1**。(提示: 实现是基于位向量、掩码的)
2. `fd_set *readfds` : 用来检查可读性的文件描述符集合。
3. `fd_set *writefds` : 用来检查可写性的文件描述符集合。
4. `fd_set *exceptfds` : 用来检查异常条件的文件描述符集合。
5. `struct timeval *timeout` : 指定等待的最长时间, 它可以是一个固定的时间段, 或者 `NULL`, 表示无限等待。



# 基于 I/O 多路复用的并发编程 - 编程实现

Concurrent based on I/O multiplexing - Programming implementation

```

23     while (1) {
24         ready_set = read_set;
25         Select(listenfd+1, &ready_set, NULL, NULL);
26         if (FD_ISSET(STDIN_FILENO, &ready_set))
27             command(); /* 从 stdin 读取 */
28         if (FD_ISSET(listenfd, &ready_set)) {
29             clientlen = sizeof(struct sockaddr_in);
30             connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
31             echo(connfd); /* 回显 */
32             Close(connfd);
33         }
34     }
35 }
36
37 void command(void) {
38     char buf[MAXLINE];
39     if (!Fgets(buf, MAXLINE, stdin))
40         exit(0); /* EOF */
41     printf("%s", buf); /* 回显 */
42 }

```

1. 由于 `select` 会修改 `fdset`，所以需要维护 `read_set` 和 `ready_set` 两个位向量，每次调用 `select` 之前都需要重新使用 `read_set` 设置 `ready_set`
2. 依据 `select` 的返回值，调用 `FD_ISSET` 判断是哪个文件描述符就绪然后进行对应的处理
3. 存在处理优先级问题，编码复杂
4. 存在并发粒度问题：粒度越小，编码越复杂；但粒度不够小，又会导致潜在的阻塞  
譬如，我们觉得等整个文件太久，于是将粒度改为等 1 行；但即使是等 1 行，面对恶意客户端，只给你这 1 行的一部分内容，亦会造成阻塞
5. 本质是个状态机（离散 / 编译：最喜欢的一集！）

# 基于 I/O 多路复用的并发编程 - 优劣分析

Concurrent based on I/O multiplexing - Advantages and disadvantages

为什么 I/O 多路复用更具有性能优势？

1. 无论线程切换还是进程切换，都会涉及到 **上下文切换**，而上下文切换是非常耗费资源的。
2. I/O 多路复用的并发编程，只需要一个进程，因此不存在进程切换，也就不存在上下文切换。
3. 内存使用更高效：线程和进程切换涉及到更多的内存分配和管理（栈、堆等），而 I/O 多路复用只需要为多个 I/O 操作维护少量的数据结构，**内存使用效率更高**。

# 基于 I/O 多路复用的并发编程 - 优劣分析

Concurrent based on I/O multiplexing - Advantages and disadvantages

优点:

1. I/O 多路复用技术比基于进程的设计给了程序员更多的对程序行为的控制
2. 基于 I/O 多路复用的事件驱动服务器运行在单一进程上下文中，每个逻辑流都能访问该进程的全部地址空间，便于在流之间共享数据
3. 事件驱动设计不需要进程上下文切换来调度新的流，比基于进程的设计要更高效

缺点:

1. 编码复杂，且随着并发粒度的减小，复杂性会不断上升
2. 只要某个逻辑流正在执行，其他逻辑流就不可能有进展，因此面对恶意客户端的攻击更加脆弱
3. 不能充分利用多核处理器

# 基于 I/O 多路复用的并发编程 - 示例代码分析

Concurrent based on I/O multiplexing - Example code analysis

整个代码其实最重要的就是搞懂 `pool` 这个结构体，它用于管理多个连接描述符。

```
typedef struct { /* 表示一个连接描述符池 */
    int maxfd; /* read_set 中的最大描述符 */
    fd_set read_set; /* 所有活动描述符的集合 */
    fd_set ready_set; /* 准备读取的描述符的子集 */
    int nready; /* 从 select 返回的准备好读取的描述符数量 */
    int maxi; /* client 数组中的最大索引 */
    int clientfd[FD_SETSIZE]; /* 活动描述符的集合 */
    rio_t clientrio[FD_SETSIZE]; /* 活动读取缓冲区的集合 */
} pool;
```

注意辨析：

- `maxi` 是 max index 的意思，即最大的读取描述符，用于 `check_clients` 中遍历处理
- `maxfd` 是 `read_set` 中的最大描述符，用于当做 `select` 的参数，它和 `maxi` 不一定相等，因为即使没有读取描述符，也有监听描述符
- `nready` 是当前准备好读取的描述符数量，用于在 `check_clients` 中判断是否存在需要处理的读取描述符（是否大于 0）

# 基于线程的并发编程

Concurrent based on threads

**线程**：运行在进程上下文中的逻辑流。类似于轻量化的进程。

- 线程上下文：唯一的线程 ID (TID, 进程内唯一)、**栈**、栈指针、程序计数器、通用目的寄存器、条件码
- 同一进程内的线程共享整个进程的地址空间，因此共享代码、数据、**堆**、共享库和 **打开的文件**

## 子概念

- 主线程：进程生命周期内的第一个线程
- 对等线程：由同一进程创建的线程，包括主线程
  - 对等线程可以互相杀死、互相访问对方的栈（共享在同一进程的地址空间中）
- 对等线程池：无父子关系的对等线程的集合
- 线程例程：线程的执行体，包括线程的代码和本地数据，类似于进程的 `main` 函数

# 基于线程的并发编程

Concurrent based on threads

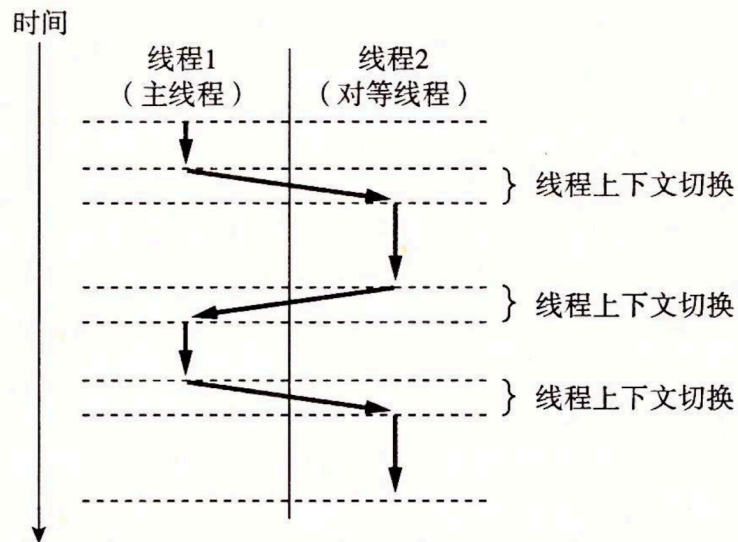
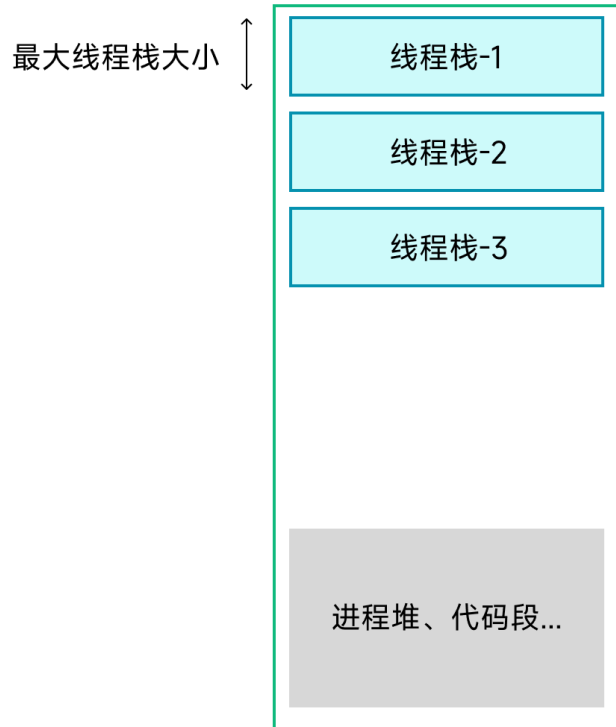


图 12-12 并发线程执行

# 基于线程的并发编程 - 编程实现

Concurrent based on threads - Programming implementation

**POSIX 线程 (Pthreads)**：一种线程 API，定义了线程的创建、同步、调度、终止等操作

```
#include "csapp.h" void *thread(void *vargp);

int main() {
    pthread_t tid;
    pthread_create(&tid, NULL, thread, NULL);
    pthread_join(tid, NULL);
    exit(0);
}

void *thread(void *vargp) {
    pthread_detach(pthread_self());
    printf("Hello, world!\n");
    return NULL;
}
```

# 基于线程的并发编程 - 编程实现

Concurrent based on threads - Programming implementation

**POSIX 线程 (Pthreads)**：一种线程 API，定义了线程的创建、同步、调度、终止等操作

```
#include "csapp.h" void *thread(void *vargp);

int main() {
    pthread_t tid;
    pthread_create(&tid, NULL, thread, NULL);
    pthread_join(tid, NULL);
    exit(0);
}

void *thread(void *vargp) {
    pthread_detach(pthread_self());
    printf("Hello, world!\n");
    return NULL;
}
```

pthread\_create：创建线程

```
int pthread_create(
    pthread_t *tidp,
    const pthread_attr_t *attr,
    void *(*start_routine)(void *),
    void *arg
);
```

1. pthread\_t \*tidp：存储线程 ID 的地方，pthread\_t 是线程 ID 结构体
2. const pthread\_attr\_t \*attr：线程属性，通常为 NULL
3. void \*(\*start\_routine)(void \*)：线程例程，注意签名：
  - 返回值：void \*，是一个指针
  - 参数：void \*，也是一个指针
4. void \*arg：线程例程的参数，亦是指针，多参数需要封装成一个结构体后传递结构体指针



# 基于线程的并发编程 - 编程实现

Concurrent based on threads - Programming implementation

**POSIX 线程 (Pthreads)**：一种线程 API，定义了线程的创建、同步、调度、终止等操作

```
#include "csapp.h" void *thread(void *vargp);

int main() {
    pthread_t tid;
    pthread_create(&tid, NULL, thread, NULL);
    pthread_join(tid, NULL);
    exit(0);
}

void *thread(void *vargp) {
    pthread_detach(pthread_self());
    printf("Hello, world!\n");
    return NULL;
}
```

`pthread_join`：显式等待线程终止，回收资源

```
int pthread_join(pthread_t tid, void **thread_return);
```

1. `pthread_t tid`：等待的线程 ID
2. `void **thread_return`：线程返回值指针存储到的地方，是一个 **指针的指针**

`pthread_join` 函数会阻塞，直到线程 `tid` 终止

它将线程例程返回的通用指针放到为 `thread_return` 指向的位置，然后回收已终止线程的内存资源。

“回收”这一操作发生在从内核态中返回到用户态时，也即在 `pthread_join` 给出返回值之前的内核态中。

# 基于线程的并发编程 - 编程实现

Concurrent based on threads - Programming implementation

**POSIX 线程 (Pthreads)**：一种线程 API，定义了线程的创建、同步、调度、终止等操作

```
#include "csapp.h" void *thread(void *vargp);

int main() {
    pthread_t tid;
    pthread_create(&tid, NULL, thread, NULL);
    pthread_join(tid, NULL);
    exit(0);
}

void *thread(void *vargp) {
    pthread_detach(pthread_self());
    printf("Hello, world!\n");
    return NULL;
}
```

`pthread_detach`：分离线程

```
int pthread_detach(pthread_t tid);
```

1. `pthread_t tid`：分离的线程 ID

分离线程不需要被其他线程回收，线程终止后 **自动回收资源**

可以使用 `pthread_self()` 获取当前线程的 ID。

# 基于线程的并发编程 - 编程实现

Concurrent based on threads - Programming implementation

```
#include <pthread.h>

pthread_once_t once_control = PTHREAD_ONCE_INIT;
int pthread_once(
    pthread_once_t *once_control,
    void (*init_routine)(void)
);

void pthread_exit(void *thread_return);

int pthread_cancel(pthread_t tid);
```

# 基于线程的并发编程 - 编程实现

Concurrent based on threads - Programming implementation

```
#include <pthread.h>

pthread_once_t once_control = PTHREAD_ONCE_INIT;
int pthread_once(
    pthread_once_t *once_control,
    void (*init_routine)(void)
);

void pthread_exit(void *thread_return);

int pthread_cancel(pthread_t tid);
```

`pthread_once` : 保证函数只被执行一次

1. `pthread_once_t *once_control` : 控制变量, 初始化时为 `PTHREAD_ONCE_INIT`
2. `void (*init_routine)(void)` : 初始化函数

在多线程环境中, 如果多个线程同时尝试初始化某个资源 (例如, 全局变量、配置文件、数据库连接等), 可能会导致竞争条件 (race condition)。

内部实现是原子操作的互斥锁, 因此可以保证函数只被执行一次。

# 基于线程的并发编程 - 编程实现

Concurrent based on threads - Programming implementation

```
#include <pthread.h>

pthread_once_t once_control = PTHREAD_ONCE_INIT;
int pthread_once(
    pthread_once_t *once_control,
    void (*init_routine)(void)
);

void pthread_exit(void *thread_return);

int pthread_cancel(pthread_t tid);
```

`pthread_exit` : 终止当前线程

1. `void *thread_return` : 线程返回值, 是个指针

- `exit` 终止当前进程, 自然会终止进程的所有线程
- 对于主线程, 等价于 `exit`, 会等待其他线程终止后再终止

# 基于线程的并发编程 - 编程实现

Concurrent based on threads - Programming implementation

```
#include <pthread.h>

pthread_once_t once_control = PTHREAD_ONCE_INIT;
int pthread_once(
    pthread_once_t *once_control,
    void (*init_routine)(void)
);

void pthread_exit(void *thread_return);

int pthread_cancel(pthread_t tid);
```

`pthread_cancel` : 取消线程

1. `pthread_t tid` : 取消的线程 ID

可以终止任意对等线程，亦可以终止自身（以  
`pthread_self()` 获取自身线程 ID）

# 基于线程的并发编程 - 并发服务器实现

Concurrent server based on threads

```

1  #include "csapp.h"
2
3  // 函数声明
4  void echo(int connfd);
5  void *thread(void *vargp);
6
7  int main(int argc, char **argv) {
8      int listenfd, *connfdp;
9      socklen_t clientlen;
10     struct sockaddr_storage clientaddr; // 客户端
11     pthread_t tid; // 线程ID
12
13     // 开启监听指定端口
14     listenfd = Open_listenfd(argv[1]);
15
16     while (1) {
17         clientlen = sizeof(struct sockaddr_storage);
18         connfdp = Malloc(sizeof(int)); // 分配空间
19         // 接受客户端连接
20         *connfdp = Accept(listenfd, (SA *)&clientaddr, &clientlen);
21         // 创建新线程处理连接
22         Pthread_create(&tid, NULL, thread, connfdp);
    
```

为什么要用 Malloc?

# 基于线程的并发编程 - 并发服务器实现

Concurrent server based on threads

```
16     while (1) {
17         clientlen = sizeof(struct sockaddr_storage);
18         connfdp = Malloc(sizeof(int)); // 分配空间
19         // 接受客户端连接
20         *connfdp = Accept(listenfd, (SA *)&clientaddr, &clientlen);
21         // 创建新线程处理连接
22         Pthread_create(&tid, NULL, thread, connfdp);
23     }
24 }
25
26 /* 线程处理函数 */
27 void *thread(void *vargp) {
28     int connfd = *((int *)vargp); // 获取连接文件描述符
29
30     Pthread_detach(pthread_self()); // 分离线程，避免僵尸线程
31     Free(vargp); // 释放传入的参数内存
32     echo(connfd); // 调用echo函数处理客户端请求
33     Close(connfd); // 关闭连接
34     return NULL; // 线程结束
35 }
```

## 为什么要用 Malloc?

- 如果直接使用 `int connfd`，则会导致所有线程共享同一个 `connfd`（位于主线程栈上）
- 会导致线程之间的 **竞争**：
  1. 主线程执行 `Accept`，将 `connfd` 设置为 3，进入 `线程 A` 创建，但还没执行 28 行赋值
  2. 由于 `pthread_create` 是非阻塞的，所以它会立即返回，主线程继续执行
  3. 主线程又一次循环，执行 `Accept`，将 `connfd` 设置为 4，进入 `线程 B` 创建
  4. `线程 A` 继续执行，将 `connfd` 设置为 4



# 基于线程的并发编程 - 并发服务器实现

Concurrent server based on threads

```
16     while (1) {
17         clientlen = sizeof(struct sockaddr_storage);
18         connfdp = Malloc(sizeof(int)); // 分配空间
19         // 接受客户端连接
20         *connfdp = Accept(listenfd, (SA *)&clientaddr, &clientlen);
21         // 创建新线程处理连接
22         Pthread_create(&tid, NULL, thread, connfdp);
23     }
24 }
25
26 /* 线程处理函数 */
27 void *thread(void *vargp) {
28     int connfd = *((int *)vargp); // 获取连接文件描述符
29
30     Pthread_detach(pthread_self()); // 分离线程，
31     Free(vargp); // 释放传入的参数内存
32     echo(connfd); // 调用echo函数处理客户端请求
33     Close(connfd); // 关闭连接
34     return NULL; // 线程结束
35 }
```

## 为什么要用 Malloc?

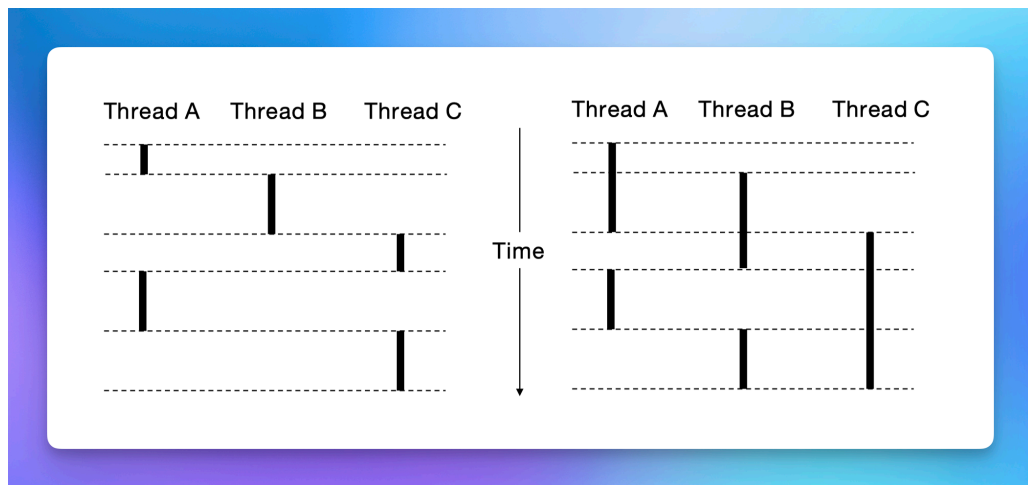
为了避免这种竞争，需要为每个线程分配独立的内存空间（从而让他们存到 **堆** 上），因此使用 `Malloc` 分配内存，在对等线程中调用 `Free` 释放内存

**注意：已连接描述符只需要关闭一次。**

# 基于线程的并发编程 - 注意事项

Concurrent based on threads - Attention

- 线程不一定是并发的
- 不能假定线程的执行顺序
- 对于共享变量，需要使用互斥量（`mutex`）进行同步（加锁与解锁）



# 基于线程的并发编程 - 总结

Concurrent based on threads - Summary

## 线程间的信息共享

- 整个虚拟地址空间都是共享的
  - 代码，堆，共享库，打开文件
  - 可以方便地访问其他线程的数据
- 每个线程有自己的栈，但是可以通过指针访问其他线程的栈

## 变量

- 全局变量 global / 本地静态变量 local static: 所有线程共享，只有一份（本来对于整个进程而言就只有一份）
- 本地自动变量（栈上的）：每个线程一份，存储在各自的线程栈中
- 共享变量：一个变量被同一进程中的不同线程使用，会导致并发问题

# 共享变量

Shared variable

**共享变量：**它的一个实例被多个线程引用。

例如：示例程序中的变量 `cnt` 是共享的，`myid` 不是共享的。

但是，本地自动变量 `msgs` 也是可以被共享的。

理解一个变量是否被共享，重点在于看它存在哪里：

- 若存在于线程上下文里，一般是不共享的，除非传递了一个指向他的指针
- 若在共享的进程数据结构（全局变量的 `data` 段、堆）里，一般就是共享的

```
void *thread(void *vargp);
char **ptr; /* 全局变量 */
int main()
{
    int i;
    pthread_t tid;
    char *msgs[N] = {
        "Hello from foo", // 来自 foo 的问候
        "Hello from bar"  // 来自 bar 的问候
    };
    ptr = msgs;
    for (i = 0; i < N; i++)
        Pthread_create(&tid, NULL, thread, (void *)i);
    Pthread_exit(NULL);
}

void *thread(void *vargp)
{
    int myid = (int)vargp;
    static int cnt = 0;
```

# 基于线程的并发编程 - 总结

Concurrent based on threads - Summary

`volatile` 关键字：防止编译器优化，确保每次访问变量时都从内存中读取

- 只对变量有效，对函数、类无效
- 不保证操作的原子性，如 `i++` 不是原子操作，它实际上是编译为先读取 `i`，然后加 1，再写回 `i`

如何理解：每次访问变量时都从内存中读取？

回忆：**寄存器也是线程上下文的一部分。**

- 对于 `i++`，**不是** 读取、加 1、写回 的时候都去内存中取，只有读取的时候会强制从内存中取、写回的时候会强制写回内存
- 举例：循环体内 `i++`，`volatile` 作用于两次循环之间，强制要求取内存读取两次 `i`，而不是在第二次使用第一次 `load` 进来后的寄存器

# 同步

## Synchronization

**竞争：**多个线程同时访问 / 修改共享资源，可能会引发一些问题

- 原因：线程共享地址空间，不同线程可能同时操作同一变量

**原子：**一旦开始就无法被打断的操作

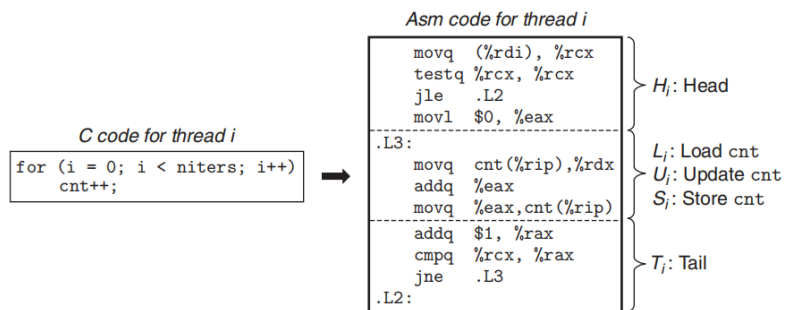
- 一行代码可能翻译成若干行汇编代码，例：`i++` 翻译成汇编 `mov, add, mov`，一般的变量不具有原子性

**死锁：**线程竞争可能导致的一种问题

- A 进行操作 a 的前提是 B 进行操作 b, B 进行操作 b 的前提是 A 进行操作 a
- 例: 开门的钥匙被锁在门里面
- **判断：**进度图 + 临界区

# 同步

## Synchronization



| (a) Correct ordering |        |        |                   |                   |     | (b) Incorrect ordering |        |        |                   |                   |     |
|----------------------|--------|--------|-------------------|-------------------|-----|------------------------|--------|--------|-------------------|-------------------|-----|
| Step                 | Thread | Instr. | %rdx <sub>1</sub> | %rdx <sub>2</sub> | cnt | Step                   | Thread | Instr. | %rdx <sub>1</sub> | %rdx <sub>2</sub> | cnt |
| 1                    | 1      | $H_1$  | —                 | —                 | 0   | 1                      | 1      | $H_1$  | —                 | —                 | 0   |
| 2                    | 1      | $L_1$  | 0                 | —                 | 0   | 2                      | 1      | $L_1$  | 0                 | —                 | 0   |
| 3                    | 1      | $U_1$  | 1                 | —                 | 0   | 3                      | 1      | $U_1$  | 1                 | —                 | 0   |
| 4                    | 1      | $S_1$  | 1                 | —                 | 1   | 4                      | 2      | $H_2$  | —                 | —                 | 0   |
| 5                    | 2      | $H_2$  | —                 | —                 | 1   | 5                      | 2      | $L_2$  | —                 | 0                 | 0   |
| 6                    | 2      | $L_2$  | —                 | 1                 | 1   | 6                      | 1      | $S_1$  | 1                 | —                 | 1   |
| 7                    | 2      | $U_2$  | —                 | 2                 | 1   | 7                      | 1      | $T_1$  | 1                 | —                 | 1   |
| 8                    | 2      | $S_2$  | —                 | 2                 | 2   | 8                      | 2      | $U_2$  | —                 | 1                 | 1   |
| 9                    | 2      | $T_2$  | —                 | 2                 | 2   | 9                      | 2      | $S_2$  | —                 | 1                 | 1   |
| 10                   | 1      | $T_1$  | 1                 | —                 | 2   | 10                     | 2      | $T_2$  | —                 | 1                 | 1   |

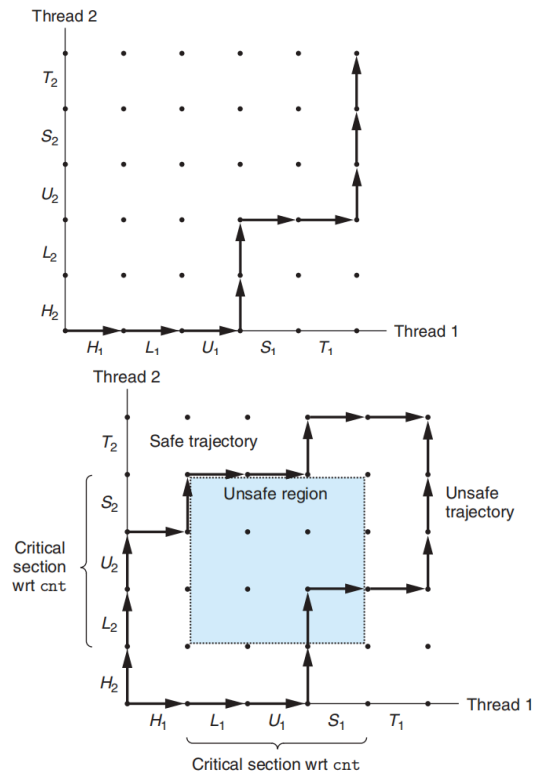
Figure 12.18 Instruction orderings for the first loop iteration in badcnt.c.

- L、U、S涉及对共享变量的操作

# 进度图

## Progress Graphs

- 每个点代表某个进程完成了某个操作
- 只有向右和向上的轨迹是合法的
- 操作共享变量的操作构成了一个临界区
- 每个线程在执行它的临界区中指令时，拥有对共享变量互斥的访问
- 两个临界区的交集形成的状态空间称为不安全区，**不包括交界的状态**
- 安全轨迹线与不安全轨迹线





# 信号量

## Semaphore

**信号量**：一种具有 **非负整数值的全局变量**，只能由两种特殊的操作处理（P 和 V）。

其目的是 **保证互斥访问**，从而保证线程化程序正确执行，禁止不安全的执行轨迹。

### P(s) 操作

如果  $s$  是非零的，那么 P 将  $s$  减 1，并且立即返回。

1. 如果  $s$  为零，则 **挂起进程**，直到  $s$  变为非零，并且该进程被一个 V 操作重启。
2. 在重启之后，P 操作将  $s$  减 1，并将控制返回给调用者。

### V(s) 操作

V 操作将  $s$  加 1，若会导致超过上限，则无事发生

如果有任何进程阻塞在 P 操作等待  $s$  变成非零，那么 V 操作会 **随机** 重启这些进程中的一个，然后该进程将  $s$  减 1，完成它的 P 操作。

V 操作是 **非阻塞** 的，有没有 P 在等对 V 无所谓。

**一个信号量可以约束至多一个边界情况（信号量非负），若想要约束两边，则需要两个信号量。**

# 信号量使用注意事项

## Semaphore usage notes

### P 的顺序很重要（不合适的话可能引发死锁）

原因：P 操作要求信号量  $s > 0$ ，这一条件不一定总成立

### V 的顺序无所谓（虽然顺序不合适可能导致效率问题，但是不会死锁）

原因：V 操作没有前提要求，只是给信号量  $s + 1$ ，总是能进行的

### 死锁

- 简单粗暴地判断：两个线程，一个先  $P(A)$  后  $P(B)$ ，一个先  $P(B)$  后  $P(A)$ ，（中间无释放，没有用其他信号量分隔），必然死锁
- 若  $P$  操作持有顺序和  $V$  操作释放顺序倒序，即构成  $P(s_1) \rightarrow P(s_2) \rightarrow V(s_2) \rightarrow V(s_1)$ ，则一般不会死锁

# 用信号量解决竞争

Use semaphore to solve race condition

```
volatile long cnt = 0; /* 计数器 */
sem_t mutex;          /* 保护计数器的信号量 */

Sem_init(&mutex, 0, 1); /* 初始化信号量，初值为1 */

for (i = 0; i < niters; i++) {
    P(&mutex);          /* 加锁 */
    cnt++;
    V(&mutex);          /* 解锁 */
}
```

实际上是通过信号量 `mutex` 来约束了同时访问 `cnt` 的线程数量  $\leq 1$ ，即保证了同一时间最多只有一个线程访问 `cnt`。

实际上因为这个数量必然非负，从而约束了两端。

`cnt` 使用 `volatile` 修饰，确保每次操作它时都是最新的。

# 关于信号量

## Recap of Semaphores

1. 信号量没有“上限”这一概念，它只要求一侧，即其值非负
2. 连续多个 `V` 操作的时候，无所谓 `V` 操作的顺序，因为 `V` 操作永不阻塞
3. 做题的时候主要考虑 `P` 操作的顺序要能够保证不会死锁
4. 做题的时候可以显式将 `P` 翻译为对应变量 `--`，`V` 翻译为对应变量 `++`，观察语义，确定你的代码没有问题
5. 单个线程里 `P` 操作数不要求等于 `V` 操作数，但一整个流程过下来要求 `P` 操作数等于 `V` 操作数

# 用信号量解决竞争

Use semaphore to solve race condition

## 生产者-消费者问题

1. 生产者制造出产品，放进缓冲区
2. 消费者从缓冲区拿出产品，进行消费
3. 当缓冲区满，生产者无法继续；当缓冲区空，消费者不能继续

## 读者-写者问题

1. 一本书，有若干读者和若干写者可能访问它
2. 只要有一个写者在访问，任意其他读者/写者都不能进入
3. 只要有若干个读者在访问，任意写者都不能进入，但是其他读者可以进入

# 生产者消费者问题

Producer-consumer problem

```
typedef struct {
    int *buf;      /* 缓冲区数组 */
    int n;         /* 最大槽数 */
    int front;     /* buf[(front+1)%n] 是第一个项目 */
    int rear;      /* buf[rear%n] 是最后一个项目 */
    sem_t mutex;   /* 保护对缓冲区的访问 */
    sem_t slots;   /* 计算可用槽数 */
    sem_t items;   /* 计算可用项目数 */
} sbuf_t;

/* 创建一个空的、有界的、共享的 FIFO 缓冲区，具有 n 个槽 */
void sbuf_init(sbuf_t *sp, int n)
{
    sp->buf = Calloc(n, sizeof(int)); /* 缓冲区最多容纳 n 个项目 */
    sp->n = n;
    sp->front = sp->rear = 0; /* 如果 front == rear, 缓冲区为空 */
    Sem_init(&sp->mutex, 0, 1); /* 用于锁定的二进制信号量 */
    Sem_init(&sp->slots, 0, n); /* 初始时，缓冲区有 n 个空槽 */
    Sem_init(&sp->items, 0, 0); /* 初始时，缓冲区没有数据项 */
}
```

```
/* 清理缓冲区 sp */
void sbuf_deinit(sbuf_t *sp)
{Free(sp->buf);}

/* 将项目插入共享缓冲区 sp 的尾部 */
void sbuf_insert(sbuf_t *sp, int item)
{
    P(&sp->slots);          /* 等待可用的槽 */
    P(&sp->mutex);          /* 锁定缓冲区 */
    sp->buf[(++sp->rear)%(sp->n)] = item; /* 插入项目 */
    V(&sp->mutex);          /* 解锁缓冲区 */
    V(&sp->items);          /* 通知有可用项目 */
}

/* 移除并返回缓冲区 sp 中的第一个项目 */
int sbuf_remove(sbuf_t *sp)
{
    int item;
    P(&sp->items);          /* 等待可用项目 */
    P(&sp->mutex);          /* 锁定缓冲区 */
    item = sp->buf[(++sp->front)%(sp->n)]; /* 移除项目 */
    V(&sp->mutex);          /* 解锁缓冲区 */
    V(&sp->slots);          /* 通知有可用槽 */
    return item;
}
```

# 生产者消费者问题

## Producer-consumer problem

```

1  #include "csapp.h"
2  #include "sbuf.h"
3
4  typedef struct {
5      int *buf;      /* 缓冲区数组 */
6      int n;         /* 最大槽数 */
7      int front;     /* buf[(front+1)%n] 是第一个项目 */
8      int rear;      /* buf[rear%n] 是最后一个项目 */
9      sem_t mutex;   /* 保护对缓冲区的访问 */
10     sem_t slots;    /* 计算可用槽数 */
11     sem_t items;    /* 计算可用项目数 */
12 } sbuf_t;
13
14 /* 创建一个空的、有界的、共享的 FIFO 缓冲区，具有 n 个槽 */
15 void sbuf_init(sbuf_t *sp, int n)
16 {
17     sp->buf = Calloc(n, sizeof(int)); /* 缓冲区最多容纳 n 个项目 */
18     sp->n = n;
19     sp->front = sp->rear = 0; /* 如果 front == rear, 缓冲区为空 */
20     Sem_init(&sp->mutex, 0, 1); /* 用于锁定的二进制信号量 */
21     Sem_init(&sp->slots, 0, n); /* 初始时，缓冲区有 n 个空槽 */
22     Sem_init(&sp->items, 0, 0); /* 初始时，缓冲区没有数据项 */

```

1. 使用 `slots` 信号量的非负性，约束了缓冲区中项目数量  $\leq n$
2. 使用 `items` 信号量的非负性，约束了缓冲区中项目数量  $\geq 0$
3. 使用 `mutex` 信号量，约束了对全局变量缓冲区的访问



# 生产者消费者问题

## Producer-consumer problem

```

1  #include "csapp.h"
2  #include "sbuf.h"
3
4  typedef struct {
5      int *buf;      /* 缓冲区数组 */
6      int n;         /* 最大槽数 */
7      int front;     /* buf[(front+1)%n] 是第一个项目 */
8      int rear;      /* buf[rear%n] 是最后一个项目 */
9      sem_t mutex;   /* 保护对缓冲区的访问 */
10     sem_t slots;    /* 计算可用槽数 */
11     sem_t items;    /* 计算可用项目数 */
12 } sbuf_t;
13
14 /* 创建一个空的、有界的、共享的 FIFO 缓冲区，具有 n 个槽 */
15 void sbuf_init(sbuf_t *sp, int n)
16 {
17     sp->buf = Calloc(n, sizeof(int)); /* 缓冲区最多容纳 n 个项目 */
18     sp->n = n;
19     sp->front = sp->rear = 0; /* 如果 front == rear, 缓冲区为空 */
20     Sem_init(&sp->mutex, 0, 1); /* 用于锁定的二进制信号量 */
21     Sem_init(&sp->slots, 0, n); /* 初始时，缓冲区有 n 个空槽 */
22     Sem_init(&sp->items, 0, 0); /* 初始时，缓冲区没有数据项 */

```

设  $p$  表示生产者数量， $c$  表示消费者数量，而  $n$  表示以项目单位为单位的缓冲区大小。对于下面的每个场景，指出 `sbuf_insert` 和 `sbuf_remove` 中的互斥锁信号量是否是必需的。

1.  $p = 1, c = 1, n > 1$
2.  $p = 1, c = 1, n = 1$
3.  $p > 1, c > 1, n = 1$

答案：

1. 不需要 (标答有误)
2. 不需要
3. 不需要

# 第一类读者写者问题

First type of reader-writer problem

```
/* 全局变量 */
int readcnt;          /* 初始值 = 0 */
sem_t mutex, w;       /* 两者初始值均为 1 */

void reader(void)
{
    while (1) {
        P(&mutex);
        readcnt++;
        if (readcnt == 1) /* 第一个进入 */
            P(&w);
        V(&mutex);
        /* 临界区 */
        /* 读取操作 */
        P(&mutex);
        readcnt--;
        if (readcnt == 0) /* 最后一个离开 */
            V(&w);
        V(&mutex);
    }
}
```

```
void writer(void)
{
    while (1) {
        P(&w);
        /* 临界区 */
        /* 写入操作 */
        V(&w);
    }
}
```

- 要求不让读者等待，除非已经把适用对象的权限赋予一个写者。

# 第一类读者写者问题

First type of reader-writer problem

```
1  /* 全局变量 */
2  int readcnt;          /* 初始值 = 0 */
3  sem_t mutex, w;      /* 两者初始值均为 1 */
4
5  void reader(void)
6  {
7      while (1) {
8          P(&mutex);
9          readcnt++;
10         if (readcnt == 1) /* 第一个进入 */
11             P(&w);
12         V(&mutex);
13         /* 临界区 */
14         /* 读取操作 */
15         P(&mutex);
16         readcnt--;
17         if (readcnt == 0) /* 最后一个离开 */
18             V(&w);
19         V(&mutex);
20     }
21 }
22
```

- 竞争的模拟来写成了 `while` 循环（而非多线程）
- 使用 `mutex` 信号量来保护对于全局变量 `readcnt` 的访问
- 使用 `w` 信号量来保证同一时间最多只有一个写者，但是可以有多个读者（注意顺序！）
- 通过在读者内判断 `readcnt` 的数量，来决定是否作为读者全体，大发慈悲释放 `w` 锁让写者进入
- 饥饿：一个线程无限期等待，无法进展。
- 看似读者优先写者，但这个优先级很弱，**所以既可以造成读者饥饿，也可以造成写者饥饿**

# 第二类读者写者问题

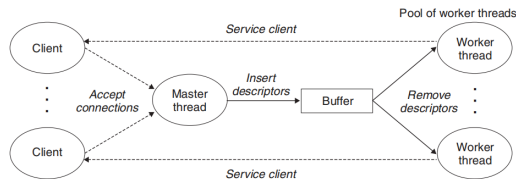
Second type of reader-writer problem

```
int readcnt, writecnt;      // Initially 0
sem_t rmutex, wmutex, r, w; // Initially 1
void reader(void)
{
    while (1) {
        P(&r);
        P(&rmutex);
        readcnt++;
        if (readcnt == 1) /* First in */
            P(&w);
        V(&rmutex);
        V(&r)
        /* Reading happens here */
        /* 此时如果有写者到来会占住r，等待w；
        而后到来的读者只能在第7行等待 */
        P(&rmutex);
        readcnt--;
        if (readcnt == 0) /* Last out */
            V(&w);
        V(&rmutex);
    }
}
```

```
void writer(void)
{
    while (1) {
        P(&wmutex);
        writecnt++;
        if (writecnt == 1) /* 第一个进入 */
            P(&r);
        V(&wmutex);
        P(&w);
        /* Writing here */
        V(&w);
        P(&wmutex);
        writecnt--;
        if (writecnt == 0) /* 最后一个离开 */
            V(&r);
        V(&wmutex);
    }
}
```

- 写者优先。在一个写者后到达的读者必须等待，即使这个写者也在等待。

# 预线程化的并发服务器



```
void *thread(void *vargp)
{
    Pthread_detach(pthread_self());
    while (1) {
        /* Remove connfd from buffer */
        int connfd = sbuf_remove(&sbuf);
        echo_cnt(connfd);    /* Service client */
        Close(connfd);
    }
}

static int byte_cnt; /* Byte counter */
static sem_t mutex; /* and the mutex that protects it */

static void init_echo_cnt(void)
{
    Sem_init(&mutex, 0, 1);
}
```

```
void echo_cnt(int connfd)
{
    int n;
    char buf[MAXLINE];
    rio_t rio;
    static pthread_once_t once = PTHREAD_ONCE_INIT;

    Pthread_once(&once, init_echo_cnt);
    Rio_readinitb(&rio, connfd);
    while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
        P(&mutex);
        byte_cnt += n;
        printf("server received %d (%d total) bytes on fd\n",
              n, byte_cnt, connfd);
        V(&mutex);
        Rio_writen(connfd, buf, n);
    }
}
```

- 参考书中代码。主要注意 `Pthread_once` 和 `pthread_once_t` 的用法

# 编写并程序

- 可以通过给线程传递一个唯一的ID指明要进行并行的哪一部分
- 减少P/V的同步次数以降低开销，例如先存在局部变量统一写（类似于cache部分）
- 当线程数大于核数时，因为上下文切换开销，运行时间可能会变长
- 加速比（强扩展）： $S_p = \frac{T_1}{T_p}$ ， $p$ 为核数， $T_k$ 为 $k$ 个核上运行时间
- 绝对加速比： $T_1$ 为程序顺序版本运行时间
- 相对加速比： $T_1$ 为程序并行版本在一个核上运行时间
- 相对加速比因同步开销会较大，但绝对加速比更难测量（需要额外编写顺序版本）
- 效率： $E_p = \frac{S_p}{p} = \frac{T_1}{pT_p}$ ，并行化造成开销的衡量
- 弱扩展：增加处理器数量的同时，增加问题规模。此时加速比和效率被表达为单位时间完成的工作总量

# 线程安全函数

## Thread-safe functions

**线程安全**：一个函数被多个并发线程反复地调用时，它会一直产生正确的结果。

回忆：线程之间存在竞争关系，在没有使用信号量等同步机制时，你不能假定其执行顺序。

四类线程不安全函数：

1. 第 1 类：不保护全局变量
2. 第 2 类：保持跨越多个调用的状态，也即返回结果强相关于调用顺序
3. 第 3 类：返回指向静态变量（`static`）的指针
4. 第 4 类：调用线程不安全的函数

# 第1类线程不安全函数

## Class 1 Thread-unsafe functions

### 第1类：不保护全局变量

- 多线程并行时，**全局变量可能被多个线程同时修改**，导致结果不正确
- 可以使用信号量来保护全局变量，使得每次只有一个线程可以访问它

```
int counter = 0;

void increment(void) {
    counter++;
}
```



## 第 2 类线程不安全函数

### Class 2 Thread-unsafe functions

第 2 类：保持跨越多个调用的状态，也即返回结果强相关于调用顺序

- `next_random` 同时是第 1 类和第 2 类线程不安全函数，因为 `next_seed` 是全局变量，且跨越多个调用的状态
- `next_random_2` 是第 2 类线程不安全函数，虽然保护了对于全局变量 `next_seed` 的访问，但是 `next_seed` 是跨越多个调用的状态，**即下一次调用返回结果依赖于从现在到那时之间是否有其他线程调用**
- 注意，`static` 声明的变量在函数内共享

```
unsigned next_seed = 1;
sem_t mutex;

/* 返回一个随机数，书上版本 */
unsigned next_random(void) {
    next_seed = next_seed * 1103515245 + 12345;
    return (unsigned int)(next_seed / 65536) % 32768;
}

/* 返回一个随机数，修改后版本 */
unsigned next_random_2(void) {
    P(&mutex);
    static unsigned next_seed_2 = 1;
    next_seed_2 = next_seed_2 * 1103515245 + 12345;
    V(&mutex);
    return (unsigned int)(next_seed_2 / 65536) % 32768;
}
```

## 第 3 类线程不安全函数

### Class 3 Thread-unsafe functions

第 3 类：返回指向静态变量（`static`）的指针

- 和第 2 类一样，你这个结果正确与否取决于在调用它得到结果到你使用它之间，是否有其他线程调用同函数
- 这也是一个第 1 类线程不安全函数，因为 `static` 变量是全局变量，这里没保护
- 若有，则此 `static` 由于在函数内共享（进而在线程间共享），所以可能会被其他线程修改

```
char* ctime(const time_t* timer) {  
    // ctime 总是返回一个长度为 26 的固定字符串  
    static char buf[26];  
    struct tm* tm_info = localtime(timer);  
    strftime(buf, 26, "%a %b %d %H:%M:%S %Y\n", tm_info);  
    return buf;  
}
```

# 第 3 类线程不安全函数

## Class 3 Thread-unsafe functions

第 3 类：返回指向静态变量（`static`）的指针

解决方法：

- 核心：设法使得结果对于每个线程私有、独立
- 做法：
  - `malloc` 分配堆上内存，使结果对线程唯一
  - 传递存放结果处指针，使结果完全独立可控
  - 

```
char *ctime_ts(const time_t *timep, char *privatep)
{
    char *sharedp;
    P(&mutex);
    // 获取时间字符串
    sharedp = ctime(timep);
    // 将共享的字符串复制到私有空间
    strcpy(privatep, sharedp);
    V(&mutex);
    // 返回私有空间的指针
    return privatep;
}
```

# 第 4 类线程不安全函数

## Class 4 Thread-unsafe functions

### 第 4 类：调用线程不安全的函数

- 但是不一定调用了就不行
- 比如第 1 类、第 3 类很多情况下加个锁就可以了
  - 使用前: `P(&mutex)`
  - 使用后: `V(&mutex)`

```
char* ctime(const time_t* timer) {  
    // ctime 总是返回一个长度为 26 的固定字符串  
    static char buf[26];  
    struct tm* tm_info = localtime(timer);  
    strftime(buf, 26, "%a %b %d %H:%M:%S %Y\n", tm_info);  
    return buf;  
}  
  
void safe_ctime(){  
    P(&mutex);  
    char* result = ctime(NULL);  
    // 使用 result  
    V(&mutex);  
}
```

# 可重入性

## Reentrant functions

**可重入函数**：被多个线程调用时，不会引用任何共享数据。

- 显式可重入：完全使用本地自动栈变量，在线程上下文切换时能保证切换回来的时候“一切如初”
- 隐式可重入：要共享的东西使用指针传递，小心处理，保证对于线程是唯一的，**这使得这个数据不再是线程间共享的**

```
/* 显式可重入 */
/* 自己玩自己的，别的线程和我无关 */
void reentrant_function() {
    int a = 1;
    int b = 1;
    for (int i = 0; i < 100; i++) {
        int tmp = a;
        a = a + b;
        b = tmp;
    }
    return a;
}
```

```
/* 寄 */
unsigned next_seed = 1;
unsigned rand(void)
{
    next_seed = next_seed * 1103515245 + 12345;
    return (unsigned)(next_seed >> 16) % 32768;
}

/* 隐式可重入 */
unsigned int rand_r(unsigned int *nextp)
{
    // 好好维护指针，那么 nextp 就不会共享
    *nextp = *nextp * 1103515245 + 12345;
    return (unsigned int)(*nextp / 65536) % 32768;
}
```

# 回顾

## Recap

- 线程安全：一个函数被多个并发线程反复地调用时，它会一直产生正确的结果
  - 末尾加 `_r` 表示是函数的线程安全版本
- 可重入：被多个线程调用时，不会引用任何共享数据

线程安全但不可重入的函数：

```
int counter = 0;

void increment(void) {
    P(&mutex);
    counter++; // 引用了共享数据
    V(&mutex);
}
```

所有的函数

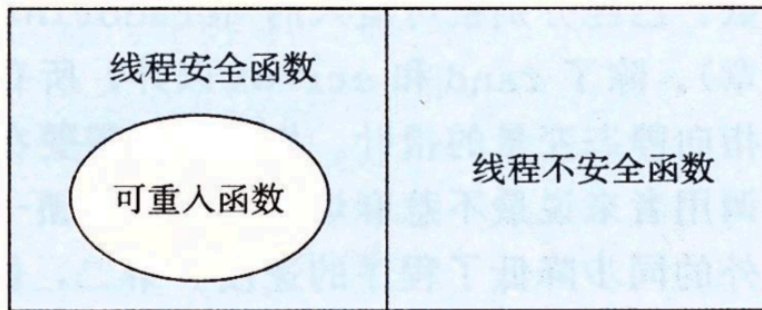


图 12-39 可重入函数、线程安全函数和线程不安全函数之间的集合关系

# 库函数

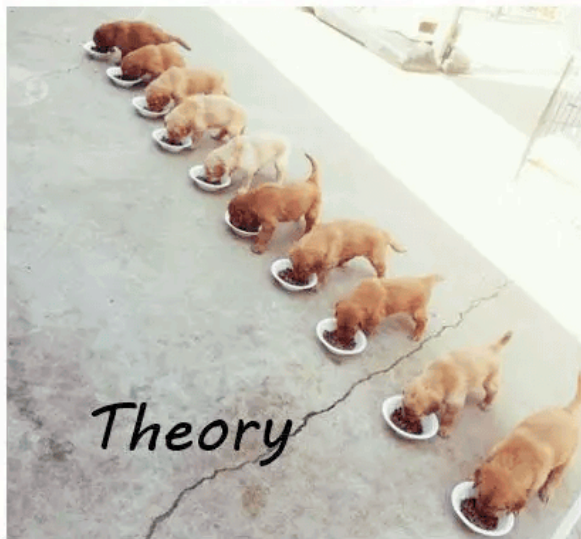
| Thread-unsafe function | Thread-unsafe class | Linux thread-safe version |
|------------------------|---------------------|---------------------------|
| rand                   | 2                   | rand_r                    |
| strtok                 | 2                   | strtok_r                  |
| asctime                | 3                   | asctime_r                 |
| ctime                  | 3                   | ctime_r                   |
| gethostbyaddr          | 3                   | gethostbyaddr_r           |
| gethostbyname          | 3                   | gethostbyname_r           |
| inet_ntoa              | 3                   | (none)                    |
| localtime              | 3                   | localtime_r               |

- 对第三类可加锁复制，但有同步开销
- 对于复杂结构需要深层复制

# 多线程编程

Multithreaded Programming

*Multithreaded programming*





# 竞争

## Races

```
void *thread(void *vargp);

int main()
{
    pthread_t tid[N];
    int i;

    for (i = 0; i < N; i++)
        Pthread_create(&tid[i], NULL, thread, &i);
    for (i = 0; i < N; i++)
        Pthread_join(tid[i], NULL);
    exit(0);
}

/* Thread routine */
void *thread(void *vargp)
{
    int myid = *((int *)vargp);
    printf("Hello from thread %d\n", myid);
    return NULL;
}
```

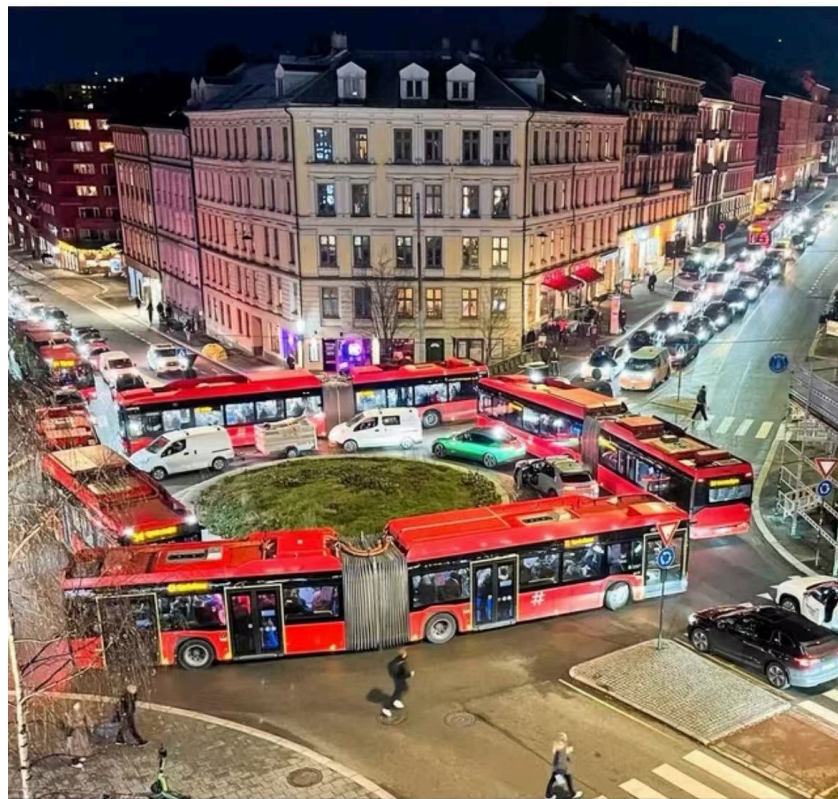
- 在主线程对 `i` 加一和对等线程对 `myid` 赋值之间有竞争，取决于对线程的调度情况

```
int main()
{
    for (i = 0; i < N; i++) {
        ptr = Malloc(sizeof(int));
        *ptr = i;
        Pthread_create(&tid[i], NULL, thread, ptr);
    }
}

/* Thread routine */
void *thread(void *vargp)
{
    int myid = *((int *)vargp);
    Free(vargp);
    printf("Hello from thread %d\n", myid);
    return NULL;
}
```

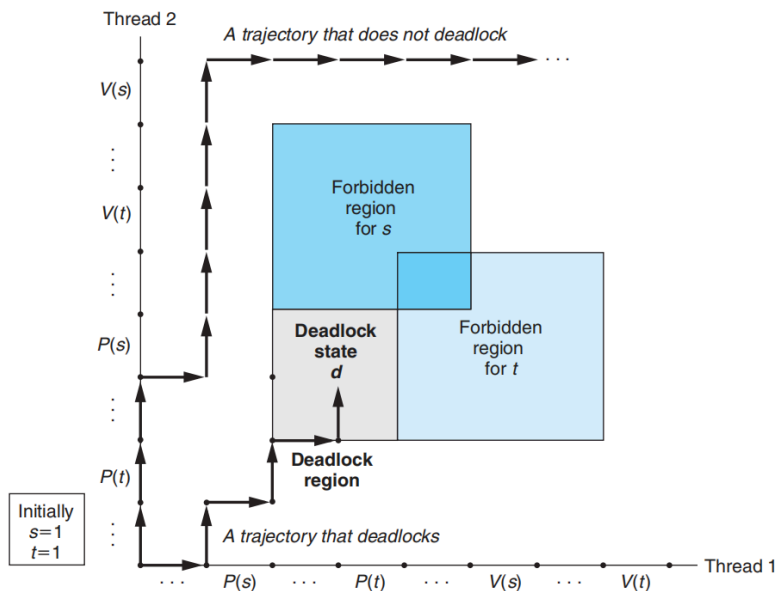
# 死锁

Deadlocks



# 死锁

## Deadlocks



- 两个信号量的禁止区域重叠，可能导致死锁区域
- 难以复现，因取决于调度
- 对于二元信号量可以有避免死锁的规则
- 给定所有互斥信号的一个全序，如果每个线程都是以同样的顺序**获得它的互斥锁**，那么这个程序就是无死锁的。
- 与书上区别：**不用看释放顺序**（勘误）

# 并发习题

## Concurrent Programming Exercises

先前第一类读者-写者问题的解答给予读者的是有些弱的优先级。

因为写者在离开它的临界区时，可能会重启一个正在等待的写者，而不是一个正在等待的读者。(Why? )

推导出一个解答，它给予读者更强的优先级，当写者离开它的临界区的时候，如果有读者正在等待的话，就总是重启一个正在等待的读者。

读者和写者竞争同一个 `w` 信号量，当一个 `V(&w)` 调用时，可能是：

- 0/1 个读者阻塞（读者之间还有 `mutex` 互锁，拿不到 `w` 时最多有 1 个读者）
- 多个写者阻塞

```

10     if (readcnt == 1) /* 第一个进入 */
11         P(&w);
12     V(&mutex);
13     /* 临界区 */
14     /* 读取操作 */
15     P(&mutex);
16     readcnt--;
17     if (readcnt == 0) /* 最后一个离开 */
18         V(&w);
19     V(&mutex);
20 }
21 }
22
23 void writer(void)
24 {
25     while (1) {
26         P(&w);
27         /* 临界区 */
28         /* 写入操作 */
29         V(&w);
30     }
31 }

```

# 并发习题

## Concurrent Programming Exercises

1. 因为弱优先问题是由读者和写者同时阻塞在 `P(&w)` 引起的，那如果我们能让读者阻塞在 `P(&w)`，而写者阻塞在其他地方，那就能解决这个问题。
2. 当 writer 释放 `V(&w)` 时，如果有多个读者和写者，则其中一个读者阻塞在 `P(&w)`，而所有写者都会阻塞在 `P(&writetry)`。
3. 不过这并不能保证读者一定能继续执行，我们需要在读者继续之前锁住这个 writer，让它没法释放 `writetry`。
4. `mutex` 正好合适，因为当有 reader 正在等待时，`mutex` 必然为 0，而这会让那个将要结束的 writer 线程阻塞，从而只有 reader 线程能继续执行下去。

```
/* 全局变量 */
int readcnt = 0;
sem_t mutex, w, writetry; /* 初始值均为 1 */
void reader() { /* reader代码不变 */
    P(&mutex);
    ++readcnt;
```

```
    if(readcnt == 1)
        P(&w);
    V(&mutex);
    /* 临界区 */
    /* 读取操作 */
    P(&mutex);
    --readcnt;
    if(readcnt == 0)
        V(&w);
    V(&mutex);
}

void writer() {
    P(&writetry);
    P(&w);
    /* 临界区*/
    /* 写入操作 */
    V(&w);
    P(&mutex);
    V(&writetry);
    V(&mutex);
}
```

# 并发习题

## Concurrent Programming Exercises

有三个线程 PA PB PC 协作工作以解决文件打印问题。

- PA 线程从磁盘读取输入缓冲区 Buff1，每执行一次读入一个记录；
- PB 将缓冲区 Buff1 的内容复制到缓冲区 Buff2，每执行一次复制一个记录；
- PC 将缓冲区 Buff2 中的内容打印出来，每执行一次打印一个记录。

缓冲区 Buff1 可以放 4 个记录，缓冲区 Buff2 可以放 8 个记录。

请设计若干信号量，给出每一个信号量的作用和初值，然后将信号量上对应的 PV 操作填写在代码中适当位置，可以留空。

```

3          // ①
4          // 从磁盘读入一个记录
5          // ②
6          // 将记录放入 Buff1
7          // ③
8      }
9  }
10 PB() {
11     while(1) {
12         // ④ // 从 Buff1 中取出一个记录
13         // ⑤ // 将记录放入 Buff2
14         // ⑥
15     }
16 }
17 PC() {
18     while(1) {
19         // ⑦
20         // 从 Buff2 中取出一个记录
21         // ⑧
22         // 打印
23     }
24 }
```

```

1  /* 信号量 */
2  sem_t empty1 /* 初值: 4 */
3  sem_t full1  /* 初值: 0 */
4  sem_t empty2 /* 初值: 8 */
5  sem_t full2  /* 初值: 0 */
6  sem_t mutex1 /* 初值: 1 */
7  sem_t mutex2 /* 初值: 1 */
8
9  PA() {
10     while(1) {
11         // NONE
12         // 从磁盘读入一个记录
13         P(&empty1); // empty1--
14         P(&mutex1);
15         // 将记录放入 Buff1, 顺序无所谓, V 操作不阻塞
16         V(&full1); // full1++
17         V(&mutex1);
18     }
19 }

```

```

PB() {
    while(1) {
        P(&full1); // full1--
        P(&mutex1);
        // 从 Buff1 中取出一个记录
        // V 操作可以挪到最后, 因为此时持有 mutex1 保证不会有 PA
        V(&mutex1);
        V(&empty1) // empty1++
        // --- //
        P(&empty2); // empty2--
        P(&mutex2);
        // 将记录放入 Buff2
        // 这里顺序无所谓, V 操作不阻塞
        V(&mutex2);
        V(&full2); // full2++
    }
}

```

```

PC() {
    while(1) {
        P(&full2); // full2--
        P(&mutex2);
        // 从 Buff2 中取出一个记录
        // 顺序无所谓, V 操作不阻塞
        V(&mutex2);
        V(&empty2); // empty2++
        // 打印
    }
}

```

# Homework Review



# Exercises

## E1

20. 下列关于 C 语言中进程模型和线程模型的说法中，错误的是：
- A. 每个线程都有它自己独立的线程上下文，包括线程 ID、程序计数器、条件码、通用目的寄存器值等
  - B. 每个线程都有自己独立的线程栈，任何线程都不能访问其他对等线程的栈空间
  - C. 不同进程之间的虚拟地址空间是独立的，但同一个进程的不同线程共享同一个虚拟地址空间
  - D. 一个线程的上下文比一个进程的上下文小得多，因此线程上下文切换要比进程上下文切换快得多

## E1

20. 下列关于 C 语言中进程模型和线程模型的说法中，错误的是：
- A. 每个线程都有它自己独立的线程上下文，包括线程 ID、程序计数器、条件码、通用目的寄存器值等
  - B. 每个线程都有自己独立的线程栈，任何线程都不能访问其他对等线程的栈空间
  - C. 不同进程之间的虚拟地址空间是独立的，但同一个进程的不同线程共享同一个虚拟地址空间
  - D. 一个线程的上下文比一个进程的上下文小得多，因此线程上下文切换要比进程上下文切换快得多

解析：B

考察 12.3 12.4 线程模型，属于简单题。B 选项，不同的线程栈是不对其他线程设防的。所以，如果一个线程以某种方式得到一个指向其他线程栈的指针，那么它就可以读写这个栈的任何部分。

## E2

22. 同一个进程内的不同线程共享以下几项？

- a) 函数体内部的静态变量
- b) 堆
- c) 文件描述符表
- d) 条件码

A. 1      B. 2      C. 3      D. 4

## E2

22. 同一个进程内的不同线程共享以下几项？

- a) 函数体内部的静态变量
- b) 堆
- c) 文件描述符表
- d) 条件码

A. 1      B. 2      C. 3      D. 4

22. C。a) 是共享的（在数据段），b) 也是共享的（栈是私有的），c) 是共享的，属于书上原话，或者从同一进程的概念中读解出，d) 是特别的上下文，是私有的。所以有 3 个。

# E3

24. 以下程序在输出有限行之后就终止了，请问最有可能的原因是（假定所有函数都正常执行）

```
#include "csapp.h"
void *thread(void *dummy){
    while (1){
        printf("hello, world!\n");
        Sleep(1);
    }
}
int main(){
    pthread_t tid;
    Pthread_create(&tid, NULL, thread, NULL);
    Sleep(3);
}
```

- A. 主线程结束必然引发所有对等线程结束
- B. 主线程结束时调用了 `_exit` 导致进程结束
- C. 主线程结束后，内核发送 `SIGKILL` 杀死进程
- D. 主线程结束后，内核观察到对等线程运行时间过长，将其杀死

# E3

24. 以下程序在输出有限行之后就终止了，请问最有可能的原因是（假定所有函数都正常执行）

```
#include "csapp.h"
void *thread(void *dummy){
    while (1){
        printf("hello, world!\n");
        Sleep(1);
    }
}
int main(){
    pthread_t tid;
    Pthread_create(&tid, NULL, thread, NULL);
    Sleep(3);
}
```

- A. 主线程结束必然引发所有对等线程结束
- B. 主线程结束时调用了 `_exit` 导致进程结束
- C. 主线程结束后，内核发送 `SIGKILL` 杀死进程
- D. 主线程结束后，内核观察到对等线程运行时间过长，将其杀死

24. B。注意主线程如果调用诸如 `pthread_exit` 的函数退出，是不会引发其他线程退出的，所以 A 为一错误描述。但是如果主线程调用了 `exit`（或者 `return`），则整个进程都会结束，自然其他线程也结束。

## E4

16. 下列关于 web 服务器向客户端提供动态内容（遵从 CGI 标准）的叙述中，哪一个是错误的？
- A. 客户端提出的 GET 请求的参数在 URI 中传递，并用“&”字符分隔文件名和多个参数
  - B. 接到客户端的请求，web 服务器创建一个子进程执行相应的 CGI 程序
  - C. 通过设置环境变量，web 服务器将 URI 中传递的参数传递给子进程
  - D. CGI 程序将动态内容重定向到与客户端关联的已连接描述符 `connfd`



## E4

16. 下列关于 web 服务器向客户端提供动态内容（遵从 CGI 标准）的叙述中，哪一个是错误的？
- A. 客户端提出的 GET 请求的参数在 URI 中传递，并用“&”字符分隔文件名和多个参数
  - B. 接到客户端的请求，web 服务器创建一个子进程执行相应的 CGI 程序
  - C. 通过设置环境变量，web 服务器将 URI 中传递的参数传递给子进程
  - D. CGI 程序将动态内容重定向到与客户端关联的已连接描述符 `connfd`

16. A。文件名和参数应该用？分隔。

# E5

19. 在以下基于线程的并发 Echo 服务器代码中，哪一个代码片段会潜在引发不正确的程序执行结果？

- A. 

```
int main(int argc, char **argv)
{
    /*假设有配对的" */
    int listenfd, connfd;

    socklen_t clientlen;
    struct sockaddr_storage clientaddr;
    pthread_t tid;
    listenfd = Open_listenfd(argv[1]);
    while (1) {
        connfd = Accept(listenfd, (SA *) &clientaddr,
            &clientlen);
        Pthread_create(&tid, NULL, thread, &connfd);
    }
    void *thread(void *vargp)
    {
        int connfd = *((int *)vargp);
        Pthread_detach(pthread_self());
        echo(connfd);
        Close(connfd);
        return NULL;
    }
```

## E5

19. 在以下基于线程的并发 Echo 服务器代码中，哪一个代码片段会潜在引发不正确的程序执行结果？

- A. 

```
int main(int argc, char **argv)
{
    /*假设有配对的" */
    int listenfd, connfd;

    socklen_t clientlen;
    struct sockaddr_storage clientaddr;
    pthread_t tid;
    listenfd = Open_listenfd(argv[1]);
    while (1) {
        connfd = Accept(listenfd, (SA *) &clientaddr,
            &clientlen);
        Pthread_create(&tid, NULL, thread, &connfd);
    }
    void *thread(void *vargp)
    {
        int connfd = *((int *)vargp);
        Pthread_detach(pthread_self());
        echo(connfd);
        Close(connfd);
        return NULL;
    }
```
- B. 

```
listenfd = Open_listenfd(argv[1]);
```
- C. 

```
while (1) {
    connfd = Accept(listenfd, (SA *) &clientaddr,
        &clientlen);
    Pthread_create(&tid, NULL, thread, &connfd);
}
```
- D. 

```
void *thread(void *vargp)
{
    int connfd = *((int *)vargp);
    Pthread_detach(pthread_self());
    echo(connfd);
    Close(connfd);
    return NULL;
}
```

19. C。否则会在对等线程的赋值和主线程的 `accept` 之间引入竞争，如果赋值在 `accept` 之后完成，描述符值就错了。

## E6

12. 使用浏览器打开网页 `www.pku.edu.cn` 的过程中，下列网络协议中，可能会被用到的网络协议有\_\_\_个

① DNS    ② TCP    ③ IP    ④ HTTP

A.    1        B.    2        C.    3        D.    4

## E6

12. 使用浏览器打开网页 `www.pku.edu.cn` 的过程中，下列网络协议中，可能会被用到的网络协议有\_\_\_个

① DNS    ② TCP    ③ IP    ④ HTTP

A. 1    B. 2    C. 3    D. 4

答案：D

考察对于网络的整体理解。按照书上涉及的内容，要解析域名必须用到 DNS 协议，要传输网页必须用到 HTTP 协议，HTTP 协议构建于 TCP/IP 协议之上，因此，所有的协议都可能会被用到。因为存在诸如 HTTPS，QUIC 等其他传输网页的协议，故加了可能两字。

## E7

14. 下列关于进程与线程的描述中，哪一个是不正确的？
- A. 一个进程可以包含多个线程
  - B. 进程中的各个线程共享进程的代码、数据、堆和栈
  - C. 进程中的各个线程拥有自己的线程上下文
  - D. 线程的上下文切换比进程的上下文切换快

## E7

14. 下列关于进程与线程的描述中，哪一个是不正确的？

- A. 一个进程可以包含多个线程
- B. 进程中的各个线程共享进程的代码、数据、堆和栈
- C. 进程中的各个线程拥有自己的线程上下文
- D. 线程的上下文切换比进程的上下文切换快

答案：B

## E8

15. 给定下列代码片段:

```
char **ptr; /* global var */
int main(int main, char *argv[]) {
    long i; pthread_t tid;
    char *msgs[2] = {" Hello from foo", " Hello from bar" };
    ptr = msgs;
    for (i = 0; i < 2; i++)
        Pthread_create(&tid, NULL, thread, (void *)i);
    Pthread_exit(NULL);}

void *thread(void *vargp)
{
    long myid = (long)vargp;
    static int cnt = 0;
    printf("[%ld]:  %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
    return NULL;
}
```

下列哪一组变量集合是对等线程 1 引用的?

- A. ptr, cnt, i.m, msgs.m, myid.p0, myid.p1
- B. ptr, cnt, msgs.m, myid.p0
- C. ptr, cnt, msgs.m, myid.p1
- D. ptr, cnt, i.m, msgs.m, myid.p1



## E8

15. 给定下列代码片段:

```
char **ptr; /* global var */
int main(int main, char *argv[]) {
    long i; pthread_t tid;
    char *msgs[2] = {" Hello from foo", " Hello from bar" };
    ptr = msgs;
    for (i = 0; i < 2; i++)
        Pthread_create(&tid, NULL, thread, (void *)i);
    Pthread_exit(NULL);}

void *thread(void *vargp)
{
    long myid = (long)vargp;
    static int cnt = 0;
    printf("[%ld]:  %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
    return NULL;
}
```

下列哪一组变量集合是对等线程 1 引用的?

- A. ptr, cnt, i.m, msgs.m, myid.p0, myid.p1
- B. ptr, cnt, msgs.m, myid.p0
- C. ptr, cnt, msgs.m, myid.p1
- D. ptr, cnt, i.m, msgs.m, myid.p1

答案: C

# 大题

## Exercise

基础：

- 网络与并发：2017期末第八题
- 读者写者问题：2019期末第七题
- 2018期末第七题

综合：

- 2020期末第六题（火锅洞）
- 生产者消费者问题：2021期末第六题
- 2016之前一些无代码的结合实际应用题（有些古早）
- 2024期末第五题（长达8页难评）

# Notices

# 期末考试时间

# 期末顺利&万事顺遂

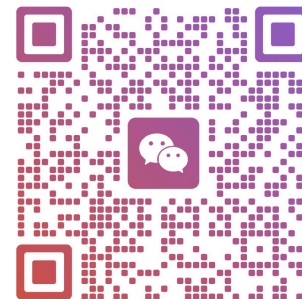
今天的问题，明天就有回答

# THANKS

Made by WEB-05

webrun@stu.pku.edu.cn

Reference: [WalkerCH]'s and [Arthals]'s presentations.



扫一扫上面的二维码图案，加我为朋友。